CEG4158 - Computer Control in Robotics

Robotics Laboratory Guide

Lab session #3

Robotic Manipulator Inverse Kinematics

P. Payeur, A. Valencia

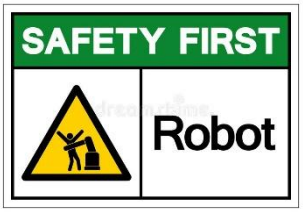
Fall 2026

School of Electrical Engineering and Computer Science

University of Ottawa

©2026

This material is protected by copyright. Reproduction, distribution, modification, or upload to any website or media of this document is strictly prohibited.



WARNING! Always play safe with robots.

1 Introduction

As you learned in Chapter 4, the forward kinematic model of a manipulator robot can be reversed, with some efforts, to develop an inverse kinematic solution. The latter stands as a set of equations placing in evidence each one of the joint coordinates of the robot, that is, an angle (θ_i) for every revolute joint or a length (d_i) for every prismatic joint. The calculation of these joint coordinates then allows the controller to precisely position and orient the end effector of the manipulator in a desired configuration, or pose, that is encoded in a specification transformation matrix, $Q_{specification}$.

The process of isolating joint coordinates from the expressions contained in the forward kinematic transformation matrix, $Q_{effector/base}$, of a given manipulator can be a daunting task and often requires a substantial amount of work and inspiration. Not all robot architectures even allow to formulate an explicit analytical inverse kinematic solution. Despite those limitations, a robot requires precise indications about the values to set on each of its revolute or prismatic joint in order to move in space and reach target configurations (e.g., to pick an object at a specific location).

An alternative way to handle inverse kinematics is the use of regression methods. With this formulation, the respective joint values are iteratively incremented or decremented to make the end effector slightly change its overall configuration, as a result of forward kinematics applied on slightly different joint values. The objective is to make the end effector progressively converge toward the desired, or specified, pose of the end effector. Therefore, joint value corrections (small angular or length variations) are computed at every iteration while monitoring the remaining error between the current Cartesian pose (position and orientation in the 3D world) of the end effector and the one that is specified for the manipulator to reach. Depending on the magnitude of the corrections, and their influence on error reduction, this process may require more or less iterations to converge, and may fail to converge to a null residual error on the end effector's pose.

The Gen3 lite robot's architecture opens the possibility to derive an approximate analytical inverse kinematic solution while assuming some simplifications. However, the process is fairly complicated and requires elaborate manipulation of subsets of equations from the forward kinematic representation. On the other hand, the CoppeliaSim simulator readily implements an iterative approach to solve the inverse

kinematics internally, given that it is designed to support a wide variety of robot architectures.

In this third laboratory session, we will not ask you to develop a formal solution to the Gen3 lite robot's inverse kinematics. We rather invite you to study the behavior of a preprogrammed analytical solution for the calculation of joint parameters and to compare it with the outcomes of an iterative inverse kinematic approach to operate the manipulator. It will also provide you with the opportunity to closely inspect the nature and impact of multiple solutions involved in inverse kinematics, a concept that was considered in this course.

This laboratory session builds upon your previous experience with the simulated Gen3 lite manipulator in Lab. #1, and upon your forward kinematic model of the robot that you developed in Lab. #2. If you previously derived an erroneous forward kinematic model, it may have an impact on achieving your goals in this laboratory session. You may need to revise your forward kinematic model accordingly.

2 Objectives

This third laboratory session provides you with an opportunity to closely study the problem of inverse kinematics when operating a 6 degrees-of-freedom manipulator robot. The specific objectives include:

1. Perform analytical inverse kinematic calculations to estimate the joint coordinates of the Kinova Gen3 lite 6 degrees-of-freedom robot;
2. Run an iterative inverse kinematic solution made available on the CoppeliaSim simulator for the Kinova Gen3 lite robot;
3. Perform an experimental evaluation of the analytical inverse kinematic solutions and their iterative counterpart in a realistic context supported by the CoppeliaSim simulator;
4. Analyze the impact of multiple solutions in inverse kinematics and design a methodology to parse these multiple solutions;
5. Analyze the relative accuracy, robustness and completeness of the solutions offered by the two inverse kinematic approaches (analytical versus iterative);
6. Expose participants to the challenges encountered when developing inverse kinematics to control a robotic manipulator ;
7. Expand your familiarity with programming functionalities used on a commercial robotic platform.

3 Experimental Protocol

Follow the steps below in order to use and study the inverse kinematic solutions that are made available to you for the Kinova Gen3 lite 6 degrees-of-freedom manipulator.

3.1 Getting started

1. Follow the procedure presented in Lab. #1 and connect to one of STE-3040 workstations.
2. Execute the necessary configuration adjustments as described in the Laboratory Guide for session #1.
3. Launch MATLAB on the workstation and set the paths properly on MATLAB.
4. Launch CoppeliaSim on the workstation.
5. IMPORTANT: To facilitate your work in this laboratory session, you should load the `Gen3Lite fk.ttt` scene, which contains the Gen3 lite manipulator model with no virtual objects, as shown in Figure 1. The scene file (.ttt file) is available in the \scenes directory of the simAPI path. Use the `loadscene()` command where appropriate in your command script.

```
v.loadscene('Gen3Lite fk','local',true)
```

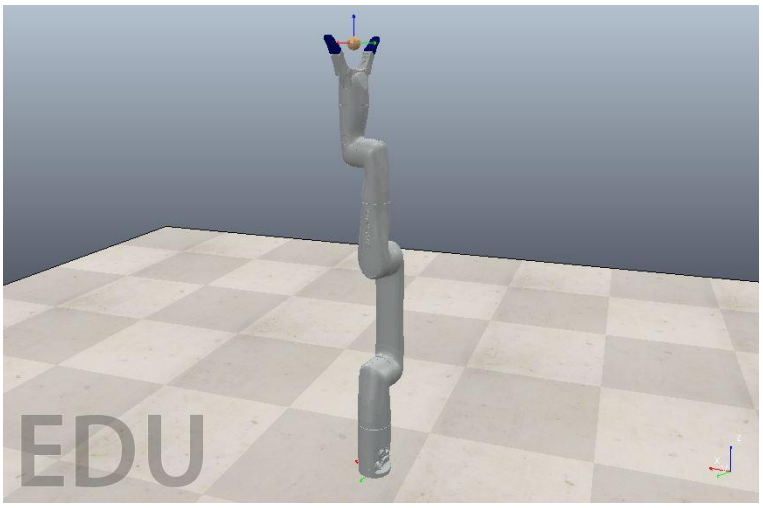


Figure 1: Scene with Gen3 lite model for inverse kinematic calculation (`Gen3Lite fk.ttt`)

3.2 Framework for the study on inverse kinematic methods

The study will involve that you perform a number of calculations using resources on MATLAB, and also running simulations on CoppeliaSim. You will then compare the results to derive a parsing strategy among multiple solutions and conduct an extensive study on the pros and cons of various inverse kinematic methodologies. While the procedure remains more flexible than in previous laboratory sessions, the depth and completeness of your investigation, the validity of your proposed approach for parsing multiple solutions, and the conclusions that you will derive form the essence of your evaluation for this activity. Figure 2 summarizes the study that you are expected to conduct. Please refer to the protocol in Section 3.4 for more details.

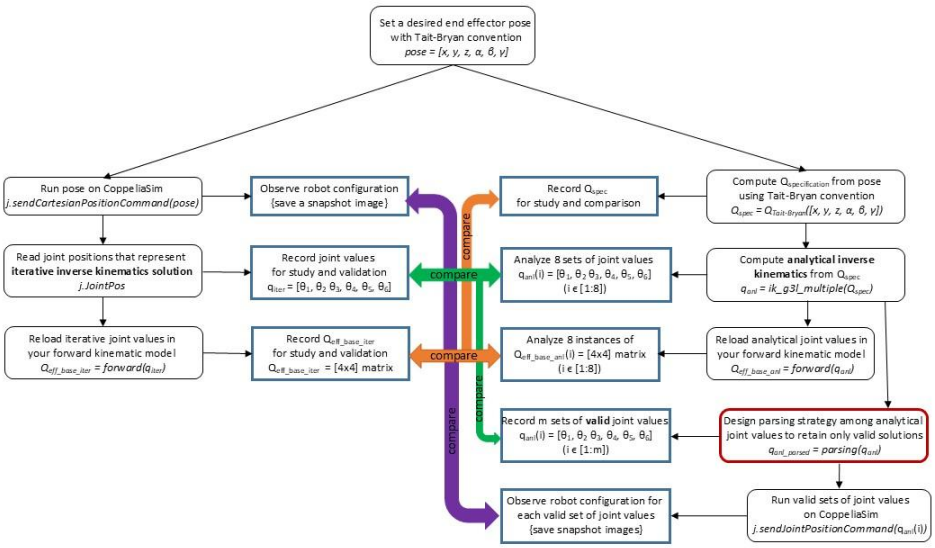


Figure 2: Overview of the case study procedure
[the red frame indicates a part that you need to design a solution for; green, orange and purple arrows indicate comparisons that you need to perform as part of the study]

3.3 Study cases

To support your investigation on inverse kinematics, it is recommended that you use three specific robot configurations, as they illustrate various complexity levels in retrieving joint parameter values. The proposed configurations are detailed in Figure 3. You are free to extend your study over additional configurations if you wish. However, your final report should cover these three cases.

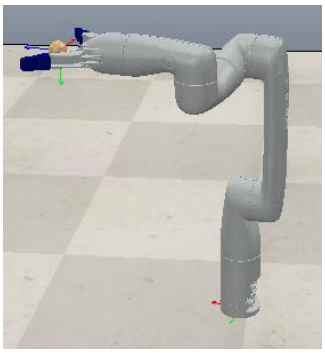
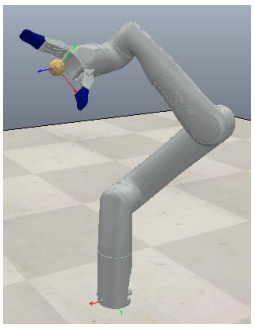
 Home configuration		
Joint angles = [0°,0°,0°,0°,0°,0°] Effector Pose (in Tait-Bryan conv.) = [0.0570, -0.0100,1.0033, 0.0000, 0.0000, 1.5708]	Joint angles = [-90°, 0°,-90°, 0°, -90°, 0°] Effector Pose (in Tait-Bryan conv.) = [0.2250, 0.2450, 0.5803, 0.0000, 1.5708, -1.5708]	Joint angles = [-45°,45°,90°,0°,-90°,0°] Effector Pose (in Tait-Bryan conv.) = [0.1701, 0.1481 , 0.5742, -1.5708, 0.7854, 2.3562]
A)	B)	C)

Figure 3: Three manipulator configurations to consider in the study

3.4 Case study protocol

3.4.1 Iterative inverse kinematics

1. Consider the robot configuration shown in Figure 3A, which represents the home configuration of the manipulator with all joints set to 0°. Given the DH parameters of the robot, the corresponding pose for the end effector with respect to R_{base} in this configuration is defined as [0.0570, -0.0100, 1.0033, 0.0000, 0.0000, 1.5708], where the notation uses the Tait-Bryan convention [x_T, y_T, z_T, α, β, γ], with angles in radians. See "*CEG4158_Lab2_Guide_rev2026.pdf*" or Section 3.4.2 below for the definition of this convention.

   On MATLAB, define a vector for the pose considered:

   ```
   pose=[0.0570, -0.0100, 1.0033, 0.0000, 0.0000, 1.5708];
   ```

2. Run the iterative inverse kinematics implemented on CoppeliaSim by setting the desired pose of the end effector on the simulated robot. The simulator will automatically attempt to make the end effector converge toward the set pose, and will indirectly estimate the necessary joint angles through a series of iterations.

   After having properly established the connection between MATLAB and CoppeliaSim, and loaded the `Gen3Lite fk` scene, run the iterative inverse kinematics with this command on MATLAB:

   ```
   j.sendCartesianPositionCommand(pose);
   ```

   and wait for the simulator to reach the target configuration. Then, observe the simulated robot configuration, save a snapshot image of it, and compare with the expected configuration, as shown in Figure 3A. Do you observe any discrepancies?

3. Read the resulting joint coordinates by interrogating the Gen3 lite instance, `j`, on MATLAB.

   ```
   q_iter = j.JointPos;
   ```

   Record the corresponding vector, `q_iter`, of 6 joint values, in radians, and convert to degrees if you wish. Inspect the results and compare with the expected joint angles, as defined in Figure 3A. Are there any discrepancies on the joint angle values?

4. To further validate the iterative inverse kinematic solution and study its precision, load the 6 joint values that you just recovered from the simulator in your forward kinematic model for the Gen3 lite robot (from laboratory #2). Make sure to load the values in the proper units, radians or degrees, depending on your implementation of the forward kinematic model. Compute the corresponding forward transformation matrix, $Q_{eff_base_iter}$, and save it for comparison later on.

3.4.2 Analytical inverse kinematics

1. Consider the same robot configuration shown in Figure 3A and use the same pose definition for the end effector.

   ```
   pose=[0.0570, -0.0100, 1.0033, 0.0000, 0.0000, 1.5708];
   ```

2. Compute the corresponding specification matrix, Q_{spec}, defining the pose of the end effector with respect to R_{base}, using the Tait-Bryan convention definition from laboratory #2.

   $Q_{spec} = Q_T(x_T, y_T, z_T)\ Q_{Rx}(\alpha)\ Q_{Ry}(\beta)\ Q_{Rz}(\gamma)$

3. Compute the analytical inverse kinematics with the `ik_g3l_multiple` method available in the simAPI main folder. This method takes as an input the specification matrix, Q_{spec}, defining the end effector's pose, computes 8 possible solutions for Gen3 lite's inverse kinematics and outputs a [8x6] matrix of joint angular values, in degrees.

   ```
   q_anl = ik_g3l_multiple(Qspec);
   ```

   The output matrix [8x6] is encoded as:

   [sol1_theta1 sol1_theta2 sol1_theta3 sol1_theta4 sol1_theta5 sol1_theta6;

   sol2_theta1 sol2_theta2 sol2_theta3 sol2_theta4 sol2_theta5 sol2_theta6;

   ...

   sol8_theta1 sol8_theta2 sol8_theta3 sol8_theta4 sol8_theta5 sol8_theta6]

   That is, one possible solution (with 6 joint angle values) per row, and 8 rows for each of the 8 possible analytical solutions.

   IMPORTANT NOTE: The solution implemented in the `ik_g3l_multiple` function assumes a simplification where theta6 (rotation of the end effector around its own axis Z_7) is frozen to 0 degree. Therefore, the matrix Q_{spec} must represent a desired configuration which is compatible with this constraint.

   Record the corresponding matrix, `q_anl_deg`, of 8x6 joint values, in degrees, and convert to radians if you wish. Inspect the results and compare with the expected joint angles, as defined in Figure 3A.

   Observe the variability in joint value combinations. Are all those solutions valid? Can they all be applied directly on the robot?

4. Analyze the validity of these multiple solutions by loading each set of 6 joint values in your forward kinematic model for the Gen3 lite robot (from

laboratory #2). Compute the 8 corresponding forward transformation matrices, $Q_{eff_base_anl}(i)$, for $i \in [1:8]$. Compare each matrix with Q_{spec} . Are all solutions from `q_anl_deg` leading back to the specification matrix, in spite of the fact that they contain different joint values? Are all those alternative solutions indicating a valid set of joint coordinates? How do you explain that different joint values may lead to a similar transformation matrix?

5. If you can identify invalid set(s) of joint angles in `q_anl_deg`, how do you propose to parse this information? In other words, how would you proceed to automatically select the valid solutions among the 8 analytical solutions available, while discarding the invalid ones? Consider three important factors for each set of 6 joint angles:

   1) Does the set of joint angles lead to reconstruct the correct Q_{spec} transformation matrix which imposes the constraint to satisfy? If not, discard the solution.

   2) Is the set of joint angles respecting the physical angular range for every one of the joint? In other words, even though the mathematical model seems to work for some solutions, can the robot execute the computed values on all of its joints? Refer to the information from the manufacturer reported on Table 1 of the "*CEG4158_Lab1_Guide_rev2026.pdf*" document about Kinova Gen3 lite's joint limits. If any computed joint value is beyond the angular range of the related joint, the corresponding solution should be discarded.

   3) Keep in mind the periodicity of the trigonometric circle that influences the outcomes of *asin*, *acos*, and *atan* functions (including *atan2*). For example, from a trigonometry perspective, an angle of -120° is the same as one of +240°. But is that also true when considering a mechanical assembly, such as a robot? Can you preserve some valid solutions by compensating for this periodicity?

   Design an appropriate parsing strategy among multiple solutions that will go through all 8 possible solutions and discard the invalid ones, therefore reducing the output of the analytical inverse kinematics to a subset of *m* , with $m \in [0:8]$, valid solutions that can be applied on the robot.

   Implement a MATLAB script that executes your proposed parsing procedure.

6. Run the parsing procedure developed above to extract the subset of *m* valid solutions. It is suggested to flag invalid solutions by assigning all their joint values to 0° (or another unique value) while preserving the other values as in the original `q_anl_deg`, after compensation for periodicity, if needed.

   Record the corresponding matrix, `q_parsed_deg`, with *m* sets of 6 joint values, and save it for comparison later on.

7. Execute the valid solutions on CoppeliaSim after parsing the output from the analytical inverse kinematics. Send a joint position command respectively for each one of the *m* sets, `q_parsed_deg(i)` , of valid joint values, for $i \in [0{:}m]$.

   ```
   j.sendJointPositionCommand(q_parsed_deg(i));
   ```

   Observe the simulated robot configuration in each case, save a snapshot image of it, and compare with the expect configuration, as shown in Figure 3A. Do you observe any discrepancies? Are all valid solutions retained after parsing leading to the proper configuration of the robot? Closely inspect the relative orientation of the end effector's reference frame (red, green, blue arrows) with respect to the orientation of the base reference frame. Does the relative orientation coincide with that of the specification, Q_{spec}? Are there still erroneous solutions that are preserved after parsing? Are there valid solutions that were erroneously discarded? Is your parsing strategy appropriate?

3.4.3 Comparative analysis

As indicated by colored arrows in the chart of Figure 2, after you obtain experimental results with both the iterative and analytical inverse kinematic methods for different configurations, perform a comparison in between the different outcomes. Here you should pay particular attention to the relative accuracy, robustness and completeness of the solutions provided respectively by the iterative versus the analytical method.

1. Green arrows: Proceed to a detailed comparison between the joint angle values obtained with the iterative inverse kinematics (`q_iter`), versus the 8 solutions obtained with the analytical inverse kinematics (`q_anl_deg`) before parsing, and versus the subset of preserved solutions after parsing. Pay attention to the unit of joint values, radians versus degrees.

   -Are the joint values corresponding with the expected ones, as shown in Figure 3A?

   -Is the precision on estimated joint values the same with both inverse kinematic methods?

   -Are all those angular values valid to control the robot?

   -What is the influence of the multiple solutions computed with analytical inverse kinematics? Is it beneficial? What challenges does it bring?

2. Orange arrows: Compare the reconstructed forward transformation matrices reconstructed respectively with the iterative inverse kinematic solution ($Q_{eff_base_iter}$), the analytical method solutions ($Q_{eff_base_anl}$), with and without solution parsing, and the desired specification matrix (Q_{spec}).

   -Do joint angle solutions all permit to retrieve the desired transformation from the base to the end effector, Q_{spec}, and therefore theoretically allow to control the robot properly?

   -Is the precision on values in these matrices the same with both inverse kinematic methods?

-Are the 8 solutions from analytical inverse kinematics entirely coherent one with each others?

3. Purple arrow: Use the visual observations of the manipulator on CoppeliaSim when the robot is given commands to reach respectively the joint angles obtained with the iterative solution, or the valid joint angles obtained with the analytical solution. Compare these configurations against the desired one shown in Figure 3A.

   -Is one of the inverse kinematic methods performing better than the other? Which one, how much better, and why?

   -Do the inverse kinematic solutions, or at least the valid ones, drive the robot to the specified configuration?

   -Are there observable errors or problems?

3.5 Expand on the comparative study

Refine your study of inverse kinematics by considering more elaborate robot configurations that may make joint parameter solutions more difficult to extract or less trivial to estimate.

For that matter, repeat the study protocol detailed in Section 3.4 while considering the two alternatives configurations presented in Figures 3B and 3C respectively.

To conduct this part of the case study, you essentially just need to reuse the code that you developed for the first case in Figure 3A, and set the end effector's pose to new coordinates, defined in Figure 3B and 3C. Then, closely analyze the results on joint coordinates, the corresponding forward transformation matrices, and overall robot configuration. This will also provide an opportunity to refine your proposed parsing strategy among multiple solutions, as more variability is involved in these test cases.

4 Laboratory Experimental Activities Evaluation

To complete this third laboratory session, **prepare a report** that will summarize your observations on inverse kinematic results with both the iterative and analytical approaches, detail your proposed parsing strategy among multiple solutions and demonstrate its validity, and formulate your conclusions about the relevance, and relative precision, robustness, and completeness of the different inverse kinematic methods.

Your report should include for each study case:

-specification transformation matrix (Q_{spec});

-experimental joint values (`q_iter`, `q_anl_deg`) in degrees without and with parsing;

-experimental forward transformation matrices ($Q_{eff_base_iter}$, $Q_{eff_base_anl}$);

-visual rendering (snapshot image) of robot configuration with valid solutions;

-detailed explanations about your proposed parsing strategy (how did you proceed?) developed in Section 3.4.2, item 5;

-discussion about the various aspects of inverse kinematics (refer to questions proposed in Section 3.4, especially the comparisons in 3.4.3). Derive your general observations. It is not necessary to repeat the discussion for each one of the three study cases. Rather synthesize your observations over all cases and present your main conclusions.

The format for the report is flexible, but present your results and observations in a structured and professional looking manner.

Upload your report (max. 10 pages including images, numerical result samples, and discussion), along with **a copy of your MATLAB script for parsing among multiple solutions**, on Brightspace, under Quizzes, "Lab 3 Evaluation". This evaluation must be performed **by team** (only one submission per team, indicate the names of all team members) and **submitted at the latest by Friday, Nov. 20th, 2026 at 11:59PM**.

5 Disconnect from Laboratory

Safely close your simulation as instructed in Lab. #1 guide.